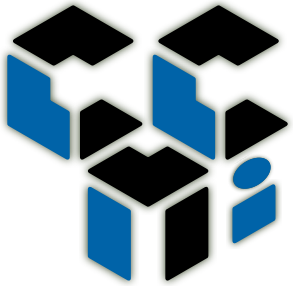
RDF a OWL: Cvičení





FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

BI-KOM: Konceptuální modelování (Cvičení #9)

Zimní semestr 2026/2027 (B261)

Plán 9. cvičení

1. Shrnutí z minula
2. Od OntoUML k OWL
3. RDF v praxi, GraphDB, SPARQL (ukázka)
4. Validace dat pomocí SHACL
5. Zadaný dataset Costy Taxi a aktivita za bod
6. Část semestrální práce s RDF
7. Úkoly na příště

Shrnutí z minula

- *TBD*

Co je potřeba z přednášky

- **RDF** = tvrzení jako triplety *(subject, predicate, object)*, zdroje identifikované URI
- **Turtle** = čitelná syntaxe RDF, kterou dnes budeme psát ručně
- **RDFS/OWL** = slovník: třídy, vlastnosti, hierarchie, omezení
- **SPARQL** = dotazování: `SELECT`, `ASK`, `CONSTRUCT`, `DESCRIBE`
- **SHACL** = validace: shapes, cíle, omezení, protokol o porušeních

---

⚠️ Dnešní cvičení je praktické. Pokud je pro vás něco z toho nové, otevřete si teď slidy z přednášky.

Od OntoUML k OWL

- Část semestrální práce s RDF vychází z modelu, který **už máte**
- Vyberte alespoň 10 tříd svého OntoUML modelu i s jejich vlastnostmi, které dávají smysl jako celek
- Přibližná korespondence (ne mechanické 1:1 mapování!):
  - «Kind», «Subkind», «Role», «Phase», ... → `owl:Class` + `rdfs:subClassOf`
  - Generalization set `{disjoint}` → `owl:disjointWith`
  - Generalization set `{complete}` → `owl:disjointUnionOf`
  - Asociace → `owl:ObjectProperty` s `rdfs:domain` a `rdfs:range`
  - Atribut → `owl:DatatypeProperty`
  - Multiplicita → `owl:Restriction` s `min` / `maxCardinality`

Od OntoUML k OWL

```
@prefix  owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix   ct: <https://kom.fit.cvut.cz/costy-taxi#> .

<https://kom.fit.cvut.cz/costy-taxi> a owl:Ontology ;
    rdfs:label "Ontologie taxislužby"@cs ;
    owl:versionInfo "1.0.0" .

ct:Person a owl:Class ; rdfs:label "Person"@en , "Osoba"@cs .
ct:Driver a owl:Class ; rdfs:subClassOf ct:Person ; rdfs:label "Řidič"@cs .
ct:Customer a owl:Class ; rdfs:subClassOf ct:Person ; rdfs:label "Zákazník"@cs .

ct:PersonalCar owl:disjointWith ct:Van .   # kindy, ne role

ct:drivenBy a owl:ObjectProperty , owl:FunctionalProperty ;
    rdfs:domain ct:TaxiRide ; rdfs:range ct:Driver .
```

Pozn.: «Role» disjunktní nejsou — tentýž člověk může být řidič i zákazník. A nezapomínejte na `rdfs:label`: ontologie, kterou nikdo nepřečte, není konceptuální model.

Ukázka: RDF v praxi

- Běžná webová stránka už RDF obsahuje, jen se musíme podívat
- validator.schema.org — jaká strukturovaná data stránka publikuje?
- search.google.com/test/rich-results — co by zobrazil vyhledávač?
- rdfa.info/play — získat celý dataset a uložit jej jako `.ttl`

- Otevřít výsledný Turtle ve VS Code a přečíst si jej — jsou to prostě triplety
- Všimněte si: tento dataset pro nás nikdo nenavrhoval, a přesto jej umíme použít

Ukázka: GraphDB

- Triple store nám dá úložiště, reasoner, SPARQL endpoint a uživatelské rozhraní
- GraphDB Free běží lokálně na `http://localhost:7200`

1. Vytvořit repozitář (`Setup` → `Repositories`)
2. Nastavit jej jako aktivní repozitář
3. Naimportovat soubor `.ttl` (`Import` → `Upload RDF files`)
4. Projít načtená tvrzení (`Explore` → `Graphs overview`)
5. Vizualizovat graf okolo jednoho zdroje (`Explore` → `Visual graph`)

Pozn.: Vizualizace grafu je nejrychlejší způsob, jak zjistit, že data nejsou propojená tak, jak si myslíte.

Ukázka: SPARQL

Spouštíme v `SPARQL` v GraphDB, dotazy stavíme postupně:

1. `SELECT` — všechny osoby v datasetu (URI a jméno)
2. `ASK` — existuje osoba s daným jménem?
3. `CONSTRUCT` — převod Schema.org na FOAF
4. `DESCRIBE` — popis osoby s daným jménem
5. `COUNT` — počet různých publikací daného autora
6. `FILTER` — publikace, jejichž název odpovídá
7. `OPTIONAL` — publikace s volitelnou alternativní URI
8. `ORDER BY` — publikace podle data a názvu
9. `GROUP BY` — počty publikací pro jednotlivé autory
10. `GROUP BY` + `HAVING` — jen kolaborativní publikace

Ukázka: SHACL

- OWL vám neřekne, že je záznam **neúplný** — SHACL ano
- Data validujeme proti **shapes**, obojí zapsané v Turtle
- Nástroje: SHACL Playground (prohlížeč) nebo `pyshacl` z příkazové řádky

```
pyshacl -s shapes.ttl -f human data.ttl
```

- Vezmeme validní dataset, spustíme validaci a pak jej **schválně rozbijeme**
- Každé porušení hlásí: *focus node*, *cestu*, *omezení* a vaši *zprávu*

Ukázka: SHACL

```
@prefix   sh: <http://www.w3.org/ns/shacl#> .
@prefix   ct: <https://kom.fit.cvut.cz/costy-taxi#> .
@prefix ctsh: <https://kom.fit.cvut.cz/costy-taxi/shapes#> .

ctsh:OrderShape a sh:NodeShape ;
    sh:targetClass ct:Order ;
    sh:property [
        sh:path ct:orderedBy ; sh:class ct:Customer ;
        sh:minCount 1 ; sh:maxCount 1 ;
        sh:message "Objednávku musí zadat právě jeden zákazník."@cs ;
    ] ;
    sh:property [
        sh:path ct:orderStatus ;
        sh:in ( "created" "cancelled" "completed" "paid" ) ;
        sh:message "Stav musí být: created, cancelled, completed nebo paid."@cs ;
    ] .
```

- `sh:in` je přesně to, co OWL neumí — v otevřeném světě není neznámý stav chybný
- `sh:message` není ozdoba — čte ji člověk, který data opravuje

Nástroje a tipy

- **Turtle pište ručně** ve VS Code (rozšíření *Stardog RDF Grammars* apod.)
  - Naučíte se tím mnohem víc než klikáním a soubory zůstanou čitelné
- Protégé se hodí na prohlížení a na spuštění reasoneru
- WebVOWL pro obrázek vaší ontologie
- Validujte průběžně, ne až na konci:
  - Syntaxe Turtle: kterýkoliv z nástrojů výše rozbitý soubor odmítne načíst
  - Konzistence: spustit reasoner v GraphDB nebo Protégé
  - Data: spustit SHACL

Pozn.: Soubor, který se nenačte, má nula bodů — a zkontrolovat to trvá 10 sekund.

Zadaný dataset: Costy Taxi (úkol 8)

Dva soubory v Turtle (Moodle), oba načtěte do **jednoho** repozitáře:

- `costy-taxi_task-08_ontology.ttl` — model (TBox): `Person` s rolemi `Driver` a `Customer`, `Vehicle` s podtřídami `PersonalCar` a `Van`, `Order` a `TaxiRide`
- `costy-taxi_task-08_data.ttl` — data (ABox): 2 řidiči, 2 zákazníci, 2 vozidla, 2 objednávky, 3 jízdy

---

- Model používá `owl:disjointWith`, `owl:inverseOf`, `FunctionalProperty`, `InverseFunctionalProperty` a jedno omezení kardinality — tedy přesně to, co chceme i v semestrální práci
- `ct:partOfOrder` v datech **nenajdete**: odvodí jej reasoner z `owl:inverseOf`, bez reasoneru vrátí dotaz na něj prázdný výsledek

Aktivita za bod: úkol 8

1. Napište **2 SPARQL dotazy** — alespoň jeden musí použít `FILTER`, `OPTIONAL` nebo agregaci; u každého uveďte účel a výsledek
2. Napište **1 SHACL shape** s alespoň dvěma omezeními a smysluplnou `sh:message`
3. Zvalidujte data tak, jak jsou, pak je **schválně rozbijte** a přiložte protokol o porušení

```
pyshacl -s shape.ttl -e costy-taxi_task-08_ontology.ttl -f human costy-taxi_task-08_data.ttl
```

Pozn.: Prázdný výsledek není hotový dotaz. A shape, který jen opakuje ontologii, nevaliduje nic.

Odevzdejte soubory `.rq` / `.ttl` a výsledky v: **Aktivita / Activity #8**

Semestrální práce: část s RDF (10 bodů)

Vezměte alespoň **10 tříd svého OntoUML modelu** včetně jejich vlastností:

- **5 bodů** — model a data, jako dva soubory v Turtle
  - `ontology.ttl` — model v **RDFS/OWL**
  - `data.ttl` — instanční dataset o alespoň **100 tripletech** podle modelu
  - Bonus **+1 bod**: vyjádřit model pomocí gUFO
- **3 body** — `q1.rq`, `q2.rq`, `q3.rq` — 3 SPARQL dotazy
- **2 body** — `shape1.ttl`, `shape2.ttl` — 2 SHACL omezení
- V textu práce: zvolený fragment a proč, tabulka použitých konstruktů, každý dotaz s účelem a výsledkem, každá shape s porušením, které odhalí

Semestrální práce: požadavky

- **Model**: alespoň 10 tříd a nad rámec pouhé hierarchie musí použít `owl:disjointWith`, jedno omezení kardinality, jednu `ObjectProperty` s `domain` a `range`, jednu charakteristiku vlastnosti (`inverseOf` / `Transitive` / `Functional`), jednu `DatatypeProperty` a `rdfs:label` u každé třídy
  - Ontologie má vlastní IRI a metadata (název, verzi, autora)
- **Dataset**: alespoň 100 tripletů, instancuje každou třídu, používá každou vlastnost alespoň jednou, načte se bez chyb a je **konzistentní pod reasonerem**
- **Dotazy**: jeden `CONSTRUCT` nebo `DESCRIBE`, jeden s agregací, jeden s `FILTER` nebo `OPTIONAL` — každý vrací neprázdný výsledek
- **Shapes**: jedna z nich musí zachycovat **OCL omezení z 1. části**

Semestrální práce: bonus a časté chyby

- **Bonus (+1 bod)**: vyjádřit model pomocí gUFO a vysvětlit, v čem se liší od prostého RDFS/OWL
- Dataset **může být vygenerovaný AI** — ale zkontrolujte jej, je za něj hodnocení a musíte jej obhájit
- Časté chyby, kterým se vyhnout:
  - Model, který je jen hierarchií tříd — bez vlastností a bez omezení
  - 100 tripletů téhož opakovaného vzoru, které se většiny modelu vůbec nedotknou
  - Tři variace téhož triviálního `SELECT`
  - Shapes, které opakují to, co už říká OWL, místo aby validovaly data
  - Snímky obrazovky místo souborů
  - Vymyšlená URI kolidující s reálnými slovníky (používejte vlastní jmenný prostor!)

Úkoly na příště

- Pracujte na 2. části semestrální práce (BPMN + RDF)
- Vyzkoušejte si nástroje teď, ne večer před odevzdáním:
  - Nainstalovat GraphDB, načíst libovolný `.ttl`, spustit jeden dotaz
- Příští přednáška a cvičení: BPMN a modelování procesů

Konec prezentace